Desenvolvimento do Modelo de Machine Learning

Confecção do Notebook
- 1. Análise inicial dos dados
- 2. Visualização dos dados
- 3. Importância das features
- 4. Cross-validation
- 5. Treinamento do baseline

Desenvolvimento dos pipelines de treinamento
- Pipeline Linear
- Pipeline Tree-based
- Treinamento dos modelos

Feature mais importantes

Escolha do modelo

Primeiramente, dado o objetivo do case, que é treinar um modelo de Machine Learning para conseguir predizer o preço das casas, estamos diante de uma tarefa de regressão.

Levando isto em consideração, a proposta é treinar um modelo baseline, `LinearRegression`, para servir de comparação para o treinamento de modelos que, em geral, produzem resultados melhor em dados tabulares. São os modelos `RandomForest`, `GradientBoosting` e `XGBoost`, todos baseados em árvores de decisão.

O **Linear Regression** é mais sensível às características do dataset quando comparado a modelos baseados em árvores, que são menos afetados por dimensionalidade, presença de outliers e multicolinearidade, além de exigirem menos etapas de feature selection. Devido à necessidade dessas etapas de pré-processamento, foi desenvolvido um notebook para exploração dos dados, visualização, feature engineering e treinamento do baseline, fornecendo apoio à construção posterior dos pipelines de treinamento.

Confecção do Notebook

- Arquivo: `models/eda-visualization-feature-sel-eng-baseline.ipynb`

O notebook foi estruturado nas seguintes etapas:

1. Análise inicial dos dados

Incluiu:

- verificação do número de linhas e colunas;
- identificação dos tipos de dados;
- interpretação dos atributos (numéricos, categóricos, geográficos e temporais);
- verificação da presença de valores nulos;

Também foram criadas novas features:

- **yr_month**: combinação de `year` e `month`, derivados de `date`;
- **house_age**: substituição de `yr_built`, representando diretamente a idade do imóvel;
- **was_renovated**: transformação binária de `yr_renovated`, que possuía predominância de zeros;

Algumas features foram removidas:

- `id`, por ser apenas identificador;
- `zipcode`, após o merge com dados demográficos;
- `date`, `year`, `month`, `yr_built` e `yr_renovated`, após criação das novas variáveis;
- colunas absolutas do dataset demográfico, mantendo apenas valores percentuais;

2. Visualização dos dados

Foram utilizados:

- histogramas para análise de distribuição das variáveis;
- gráficos exploratórios relacionando preço com variáveis socioeconômicas;
- boxplots para análise de outliers;
- matriz de correlação entre features e variável alvo;

Observou-se assimetria à direita nas distribuições, motivando a aplicação de **log-transform** em algumas variáveis - quando aplicável.

Também foi identificado que:

- imóveis de maior valor concentram-se em determinados *zip codes*;
- regiões com maior escolaridade apresentam maior valorização imobiliária;

Testes mostraram que a remoção de outliers reduziu o desempenho do modelo baseline, indicando que os outliers apresentam informações relevantes associadas a casos muitos caras.

3. Importância das features

Foi utilizado **LassoCV** para estimar a relevância das variáveis:

- features redundantes tendem a coeficiente zero;
- apenas `per_urbn` foi descartada automaticamente pelo modelo.

As dez features mais relevantes para o LassoCV foram:

1. waterfront
2. per_bchlr
3. grade
4. medn_incm_per_prsn_amt
5. per_prfsnl
6. per_less_than_9
7. sqft_living
8. sqft_above
9. was_renovated
10. lat

4. Cross-validation

Foi realizado o cross-validation para verificar estabilidade do modelo entre diferentes *folds* de treinamento.

5. Treinamento do baseline

Treinamento com **Linear Regression** utilizando R^2, MAE e RMSE como métricas de avaliação.

Desenvolvimento dos pipelines de treinamento

Arquivo: `train.py`

Após a etapa exploratória, foram implementados pipelines automatizados de treinamento.

O processo inclui:

1. Leitura dos datasets via linha de comando;
2. Limpeza de dados e criação de features (`yr_month`, `house_age`, `was_renovated`);
3. Merge dos datasets;
4. Divisão treino/teste (80% / 20%);
5. Criação de pipelines específicos por tipo de modelo;
6. Treinamento dos modelos.

Pipeline Linear

Inclui:

- log-transform em subconjunto de features;
- scaling das variáveis não-binárias;
- seleção de features com **LassoCV**;
- transformação logarítmica da variável alvo via `TransformedTargetRegressor`

Pipeline Tree-based

Inclui:

- log-transform nas mesmas features e na variável alvo;

Treinamento dos modelos

A função `train_and_evaluate_models()` realiza:

- otimização de hiperparâmetros com **RandomizedSearchCV**;
- validação cruzada;
- armazenamento dos melhores pipelines;
- salvamento das métricas de avaliação;

Os modelos são salvo usando **joblib**.

Feature mais importantes

Para determinar as features mais importantes para o modelo, podemos utilizar o método `get_booster()` do XGBoost. A imagem abaixo mostra o resultado extraído do modelo, indicando as features mais importantes (aquelas que contribuem para a diminuição do erro):

	feature	importance
	None Count 0	None Count 0
	10 Unique values	2.6 … 39.1
0	per_bchlr	39.105324
1	per_prfsnl	20.115664
2	grade	18.923775
3	sqft_living	12.640376
4	view	7.325098
5	sqft_above	5.796853
6	lat	5.035030
7	sqft_living15	4.466350
8	waterfront	3.562164
9	medn_incm_per_prsn…	2.568623

Aqui deve ser feitas algumas observações:

- Confirma-se a suspeita de que a escolaridade média da população de uma área possui um impacto relevante da determinação do preços dos imóveis;
- Grade é uma variável ordinal que indica a qualidade da construção/casa. É um parâmetro subjetivo, ou seja, a nota depende da pessoa que está avaliando.

Isto torna a predição enviesada. Talvez fosse interessante fazer novos experimentos com o modelo removendo essa feature ou pesquisar algum tratamento que minimize sua importância.

- O pipeline foi feito incluindo quase todas as features do dataset mergeado, o que pode ter impacto na forma como o modelo é consumido, dado que, para que a predição seja feita, teríamos que ter o mesmo conjunto de atributos do treinamento;
    - Dependendo da aderência às regras de negócio, isto pode ser problemático - ex.: pode ser que o corretor ache tedioso ter que preencher tantas colunas
    - Logo, seria interessante encontrar fazer novos experimentos verificando o desempenho dos modelos em um subconjunto menor de features

Escolha do modelo

Ao total foram treinados quatro modelos:

- Regressão Linear, o baseline;
- Random Forest
- Gradient Boosting
- XGBoost

Como esperado, o XGBoost teve o melhor desempenho, superando em praticamente 0.6 pontos (R^2 score) o baseline.

O modelo é uma escolha interessante por conseguir capturar relações não-lineares e por otimizar loss e regularização, o que ajuda a evitar o overfitting. Além disso, em comparação a outros modelos, especialmente os lineares, ele é robusto a valores ausentes e outliers.

Output de `train.py`:

```
===== Treinando: LinearRegression =====
Best params: {}
CV R²: None
Test R²: 0.8427439591252617
```

```
Test MAE: 86563.06565077911
Test RMSE: 154186.41717730128

===== Treinando: RandomForest =====
Best params: {'model__regressor__n_estimators': 600, 'model__
regressor__min_samples_split': 5, 'model__regressor__min_samp
les_leaf': 1, 'model__regressor__max_features': 0.5, 'model__
regressor__max_depth': 30}
CV R²: 0.8713649309879902
Test R²: 0.8822919084101323
Test MAE: 69709.59941843436
Test RMSE: 133396.8601301588

===== Treinando: GradientBoosting =====
Best params: {'model__regressor__subsample': 0.8, 'model__reg
ressor__n_estimators': 200, 'model__regressor__min_samples_sp
lit': 10, 'model__regressor__min_samples_leaf': 4, 'model__re
gressor__max_depth': 4, 'model__regressor__learning_rate': 0.
1}
CV R²: 0.9024340730149877
Test R²: 0.901436873480241
Test MAE: 66399.8767176904
Test RMSE: 122067.3970272724

===== Treinando: XGBoost =====
Best params: {'model__regressor__subsample': 0.8, 'model__reg
ressor__reg_lambda': 5, 'model__regressor__reg_alpha': 0.1,
'model__regressor__n_estimators': 600, 'model__regressor__min
_child_weight': 1, 'model__regressor__max_depth': 3, 'model__
regressor__learning_rate': 0.1, 'model__regressor__gamma': 0,
'model__regressor__colsample_bytree': 0.6}
CV R²: 0.9054447866164905
Test R²: 0.9082229155998028
Test MAE: 64006.4294435288
Test RMSE: 117790.31318466661
```